第八届全球校园人工智能算法精英大赛 · 算法主题赛

ReproLens

开源 AI 项目复现体检

证据约束的复现就绪度审计、声明验证与最小整改智能体

作品名称	ReproLens：开源 AI 项目复现体检
参赛方向	AI 开发工具与开源协作
作品形态	开源工具 • 网页入口 • GitHub Action • CLI
参赛形式	个人参赛
代码仓库	https://github.com/czkPK/reprolens
日期	2026 年 10 月

本报告正文控制在 15 页以内；材料中不包含学校名称、学校 LOGO 或指导教师信息。

摘要

ReproLens 是一个面向开源 LLM/RAG/Agent 项目的复现就绪度审计与最小整改智能体。它将仓库中的 README、依赖、配置、prompt、模型卡、数据集卡、测试和脚本转化为可追溯的证据图，按环境、数据、模型、实验、文档和 LLM 特有风险六个维度进行评分，并生成带证据 ID 的整改计划。工具会把 README 中的可验证声明映射到对应检查项，形成“声明—证据—缺口”矩阵，同时报告证据置信度和证据覆盖率。ReproLens 支持本地 CLI、任意公开 GitHub 仓库网页入口和可复用 GitHub Action；语义层可采用可选 LLM 进行声明映射，并通过检查项目录、证据 ID 和离线回退保证输出可核验。工具进一步提供保守的 AutoFix 机制：在 staging 副本中应用修复候选，重新审计，只保留能够提升评分的修改。

一、作品概述

1. 背景与问题

开源 AI 项目公开了代码，并不等于其他人能够复现结果。常见阻塞包括：依赖版本漂移、数据和权重缺失、prompt 与模型版本未记录、评测协议不可验证、RAG 索引不可追溯、Agent 调用过程不可审计，以及许可证和来源边界不清。

现有工具大致分为论文 artifact 评审、实验管理、环境打包和人工 checklist。它们通常要求作者主动采用，或面向论文评审，缺少面向开源仓库维护者的“诊断—整改—复测”闭环。

2. 目标与作品形态

ReproLens 的目标是回答三个问题：

1. 这个开源 AI 项目能否被别人复现？
2. 如果暂时不能，最关键的缺口是什么？
3. 最小修复动作是什么，修完后是否真的变好？

作品形态包括 Python CLI、网页版、GitHub Action 和静态 HTML 报告。网页版支持粘贴任意公开 GitHub 仓库地址，固定 commit SHA 后生成报告；CLI 支持 `audit`、`--autofix`、`--explain` 和可选 `--ai-review`。

3. 核心亮点

- 使用方式感知和项目类型感知的评分；
- LLM 特有复现风险检查；
- 证据 ID 和可点击证据位置；
- README 声明到检查项的证据映射与缺口追踪；
- 高/中/低证据置信度和适用检查项覆盖率；
- 任意公开 GitHub 仓库的一键网页审计与 commit 固定；
- 可在开源项目 PR 中复用的 GitHub Action；
- LLM 语义声明映射与证据 ID 校验，异常时自动回退规则；
- 保守 AutoFix 和 staging 复测；
- 开发集与 fresh holdout 双组评估；

- 无 API 时可离线输出确定性解释。

二、需求分析与总体方案

1. 需求分析

目标用户包括开源 AI 项目维护者、尝试复现论文/模型的学生和研究者、需要评估第三方仓库的实验室和教师，以及需要快速判断仓库质量的比赛评审和开源社区用户。核心需求不是“生成一段总结”，而是获得可核验的缺口清单和可执行的整改步骤。

需求优先级：

1. 能运行、能演示；
2. 每条结论有证据；
3. 能指出最小整改动作；
4. 能复测修复效果；
5. 能接入 GitHub 协作流程。

2. 现有方案与差异化

方案类别	主要能力	主要局限	ReproLens 的差异
人工复现 checklist	按经验逐项检查	依赖个人经验，证据散落，难以复测	把检查项、证据 ID、缺口和整改动作结构化
论文 artifact 评审	判断论文附件是否可运行	偏论文场景，缺少通用开源仓库入口	直接面向 GitHub / 本地 AI 仓库
环境打包工具	固定依赖和运行环境	主要解决环境问题，不验证 README 声明	覆盖环境、数据、模型、实验、文档和 LLM 特有风险
通用代码扫描器	找代码缺陷和安全问题	不判断复现证据，不解释研发声明	使用项目类型感知的复现规则和声明验证
普通 AutoFix 工具	生成修复后直接修改文件	缺少修复后有效性验证	在隔离副本中复测，只保留提升评分的候选

3. 总体方案

本地路径或 GitHub URL
  -> 网页/CLI/Action 统一入口
  -> 公开仓库 commit 固定与浅克隆
  -> 文件盘点与项目类型识别
  -> 证据抽取（README/config/prompt/model card/tests）
  -> 证据图与适用性判断
  -> README 可验证声明抽取与检查项映射
  -> 声明状态、证据置信度与覆盖率计算
  -> 多维度评分
  -> 问题清单与整改建议
  -> AutoFix 提案 + staging 复测
  -> Markdown/HTML/JSON/解释报告

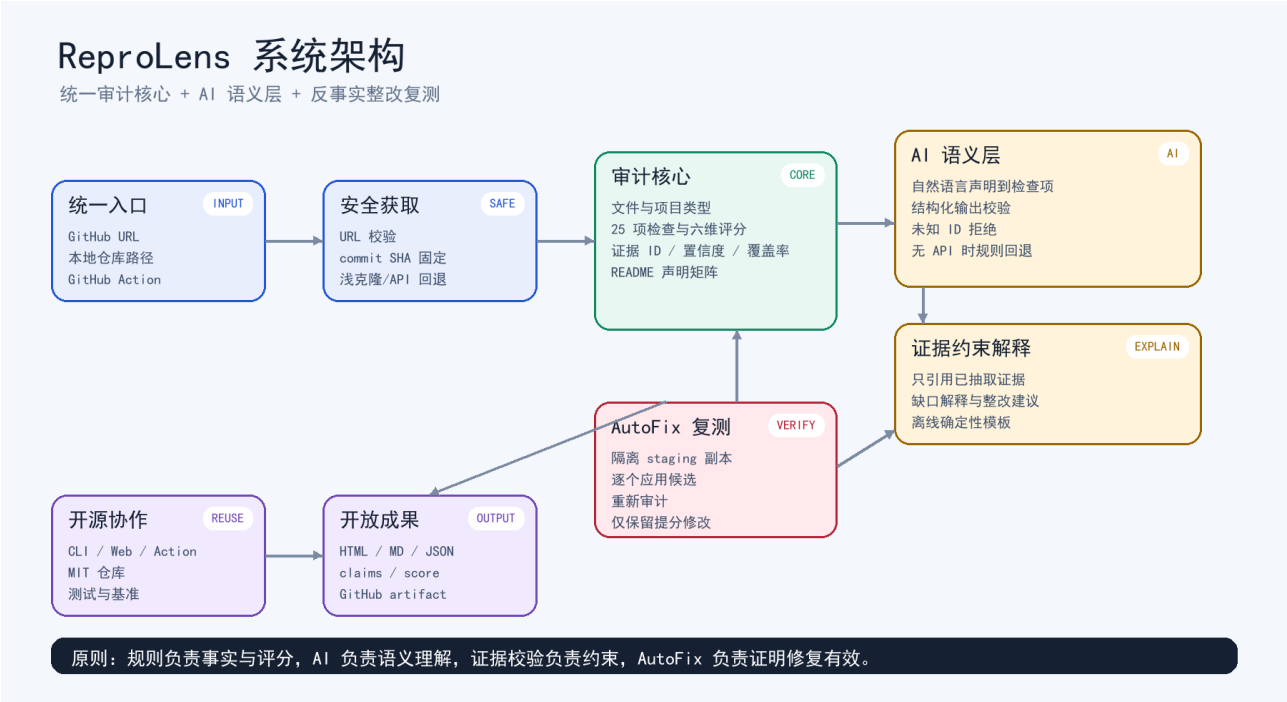


图 1 ReproLens 总体架构：统一入口、确定性审计核心、AI 语义层和 AutoFix 复测闭环。

4. 可行性与成本

- 仅使用 Python 标准库即可运行核心流程；
- 不要求训练大模型、不要求 GPU；
- 执行探针默认只做安全语法解析；
- 可选接入 OpenAI-compatible API，失败时自动回退离线模板。
- 网页端不需要安装，核心审计仅使用 Python 标准库和 Git/GitHub 公开 API。

三、AI 技术与开源资源选择

1. AI 技术作用

ReproLens 采用“确定性事实层 + AI 语义层 + 规则回退”的混合方式：

1. 规则层从 README、模型卡和数据卡中抽取候选声明、文件和证据；
2. AI 语义层把自然语言声明映射到检查项目录，解决纯关键词匹配难以处理的同义表达、复杂句和上下文；
3. AI 生成的整改建议和解释必须引用已抽取证据；
4. 校验器拒绝未知检查项和不存在的证据 ID，服务不可用时回退确定性规则。

事实校验、适用性判断、评分和复测仍由确定性规则完成；LLM 不直接决定分数，只辅助语义映射和解释。这样既利用 AI 处理非结构化文本，又避免生成式模型制造没有证据的结论。评价时应分别报告规则模式和 AI 增强模式，不能把回退结果包装成 AI 效果。

2. 开源资源

当前原型使用 Python 标准库；执行环境为 Python 3.11+；可选接入 OpenAI-compatible 模型服务；网页版使用 Git 和 GitHub 公开 API。所有软件、评测仓库、GitHub Actions 和 AI 服务的来源、版本、许可及使用边界已登记在《开源及第三方资源使用清单》中。

3. 自主开发边界

ReproLens 的扫描器、规则、评分、声明矩阵、AutoFix、网页服务、Action、基准和报告生成逻辑为本项目自主实现；第三方模型/服务用于可选语义解释，不纳入核心事实判断。AI 辅助开发环节和人工审核流程见《AI工具使用说明》。

四、设计与实现过程

1. 核心功能

- 项目类型识别：application、sdk、agent_framework、memory_framework、ml_project；
- 25 个复现检查项；
- 证据范围和占位符过滤；
- README 声明—证据—缺口矩阵，区分已验证、部分验证、未验证和不适用；
- 证据置信度（高/中/低/N/A）与适用检查项覆盖率；
- 安全语法探针，可选 `--help` 和 pytest collection；
- 任意公开 GitHub 仓库网页审计、commit 固定、clone 失败时的 GitHub API 归档回退；
- 可复用 GitHub Action，支持 PR 场景；
- HTML 证据锚点与可点击证据 ID，支持从结论跳转到文件、行号和摘录；
- 前三项优先整改摘要与可嵌入 README 的复现徽章；
- GitHub Action 自动写入 Job Summary，并输出评分和徽章路径；
- 可选 LLM 语义声明映射与结构化输出校验；
- AutoFix 提案和 staging 复测；
- Markdown、HTML、JSON 和解释报告。

2. 关键实现

- `scanner.py`：文件盘点、证据抽取、项目类型识别；
- `rules.py`：适用性和状态判断；
- `scoring.py`：维度与总分计算；
- `autofix.py`：候选生成、逐个应用、复测和自动丢弃；
- `claims.py`：声明到检查项的映射、状态汇总、证据置信度和覆盖率；
- `ai_claims.py`：LLM 语义映射、结构化输出校验和规则回退；
- `web.py`：公开 GitHub 仓库的网页审计、commit 固定、报告服务和安全限制；
- `action.yml`：GitHub 复合 Action 的安装、审计和输出约定；

- `badge.py`：复现分数徽章、机器可读徽章 JSON 和颜色分级；
- `llm.py`：证据约束解释和离线回退；
- `benchmark.py`：开发集、旧 holdout 和 fresh holdout 评估。

3. 团队分工与过程记录

本项目为个人参赛。本人负责需求界定、规则设计、代码实现、真实仓库标注、测试、报告和演示材料；AI 编程助手仅用于辅助起草代码、补充测试和整理文档，所有输出均经过运行测试、证据核对和人工修改。开发过程采用“实现核心 CLI -> 真实仓库发现误报 -> 修复规则边界 -> 增加 fresh/final holdout -> 增加声明矩阵 -> 增加网页、Action 和 AI 语义层”的迭代路线。版本迭代记录保存在 Git 历史中。

4. 开发过程

开发过程先实现静态规则和报告，再通过真实仓库发现误报，随后加入证据范围过滤、项目类型识别、占位符过滤和 AutoFix。进一步迭代中增加了声明—证据矩阵，并对 README 中的 HTML、Markdown 链接和营销式表述进行清洗与去重，避免把泛化描述误判为技术证据。随后增加网页入口、commit 固定、GitHub Action 和 AI 语义映射层。当前 25 个单元测试通过，真实网页端到端验证可完成公开仓库克隆、审计和报告生成。

五、成果展示与效果验证

1. 成果展示

演示仓库修复前 49.1 分，AutoFix 后 68.4 分，提升 19.3 分。演示仓库共抽取 2 条可验证 README 声明，证据覆盖 16/25 个适用检查项（64.0%）。真实仓库结果：

仓库	修复前	修复后
openai-python	94.9	100.0
smolagents	78.8	85.9
mem0	84.8	87.9
aider	74.8	81.0
chat-ui	44.1	62.8

补充真实仓库核验：对 `mem0` 的 1,837 个文件完成审计，抽取 8 条 README 技术声明，16/16 个适用检查项均有关联证据。该数字表示“有证据覆盖”，不等同于所有检查项都已通过。

网页端到端验证：输入 `https://github.com/huggingface/smolagents`，系统固定到 commit `96f33faaf028479119ec8d34507b47694cf14e34`，扫描 185 个文件，得到 78.8 分；AutoFix 复测后为 85.9 分（+7.1），HTML 报告、声明矩阵和评分 JSON 均可访问。



图 2 ReproLens 网页入口：粘贴任意公开 GitHub 仓库地址即可发起审计。

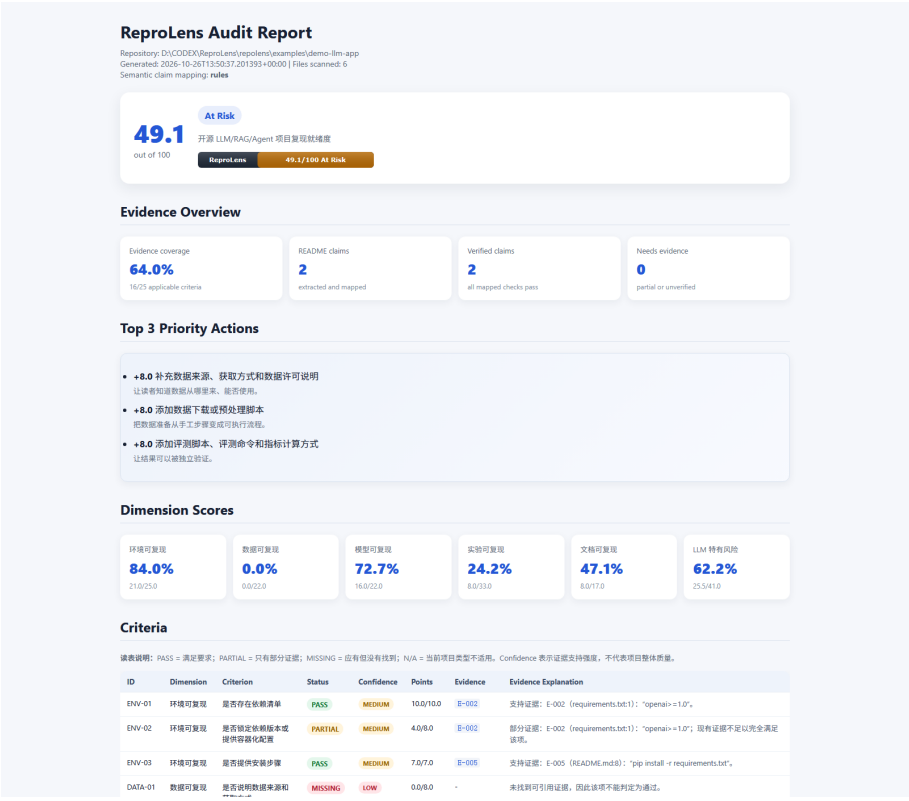


图 3 真实仓库报告：总分、证据覆盖率、优先级动作、可点击证据和来源 commit。

2. 验证方法

- 3 个开发仓库，每个 14 个关键检查项；
- 旧 holdout 3 个仓库；
- fresh holdout 2 个全新仓库；
- 人工标注 + 项目类型判断；

- exact status accuracy、Precision、Recall、F1。

3. 验证结果

开发集（含演示 fixture）：

```
56/56 exact status accuracy
```

旧 holdout（规则迭代后）：

```
38/42 exact, Precision 0.917, Recall 0.786, F1 0.846
```

Fresh holdout（未参与规则调整）：

```
26/28 exact (92.9%), Precision 1.000, Recall 0.818, F1 0.900
```

Fresh holdout（针对 RAG/Agent 证据门槛修复后复跑）：

```
27/28 exact (96.4%), Precision 1.000, Recall 1.000, F1 1.000
```

这批复跑结果不能替代初次独立评估。当前唯一 exact 差异是 open-webui 的 LLM-03 在人工标注中为 `partial`、规则输出为 `missing`，二者在缺口检测指标中都属于正类，因此 F1 未受影响。

Final holdout（最后一批应用仓库）：

```
28/28 exact (100.0%), Precision 1.000, Recall 1.000, F1 1.000 (post-fix validation)
```

Fresh holdout 的初次独立结果才是核心泛化证据，开发集 100% 不应作为泛化能力宣传；修复后复跑结果应明确标注为 post-fix re-evaluation。

4. 当前验证边界

- 已完成自动化功能测试、真实仓库端到端测试和人工标注 holdout；
- 已完成网页版真实 GitHub 仓库审计和 AutoFix 复测；
- AI 语义层的结构化输出校验和规则回退已有单元测试，但尚未获得带 API 凭据的大规模效果对比；
- 尚未完成外部用户试用和问卷，因此不宣称已获得真实用户反馈；
- 后续应补充 5—8 名开发者的试用记录和 AI 模式消融实验。

六、创新性与开放价值

1. 项目类型感知的适用性判断：SDK、框架和应用的模型/数据/实验指标不同，避免“一刀切”误报。
2. 声明—证据—缺口矩阵：不满足于“仓库里有没有某个文件”，而是检查 README 声称的能力是否被对应检查项及证据支持，并明确列出缺口。
3. 证据置信度与覆盖率：区分高/中/低证据强度，并给出适用检查项中有多少项存在可追溯证据，便于比较不同仓库的诊断完整度。
4. LLM 特有复现风险本体：模型/API 版本、prompt 版本、采样参数、tokenizer、RAG 索引、Agent trace、污染和成本。
5. 反事实最小整改：候选文件逐个加入 staging，重新评分，只保留有益修改。

6. 证据约束解释：LLM 只能基于已抽取证据生成解释，无证据时保留 Evidence Missing。
7. 开放基准：开发集、旧 holdout 和 fresh holdout 的标注、规则和结果均可复现。
8. 多入口交付：网页支持任意公开 GitHub 仓库，Action 支持开源协作自动化，CLI 支持离线复用；三种入口共享同一审计核心和证据格式。

七、合规、安全与风险说明

- 本工具不执行不可信仓库代码，默认只做语法解析；
- `--execute` 只应在受信任仓库上使用；
- 第三方资源已登记许可证和关键义务，见《开源及第三方资源使用清单》；
- 生成模板中的 TODO/TBD 不算已验证证据；
- 本工具提供工程合规检查，不构成法律意见；
- LLM 输出必须人工复核，不能替代事实判断；
- 材料中不得出现学校名称、LOGO、指导教师信息。

八、总结与展望

ReproLens 已形成“声明验证—扫描诊断—整改—复测”的完整闭环。Fresh holdout 的初次独立评估为 92.9% exact accuracy（F1 0.900），修复后复跑为 96.4%（F1 1.000）；final holdout 修复后为 100%（F1 1.000），但该批仓库参与了最后边界修复，应作为 post-fix validation 报告。声明矩阵用于解释“项目自称能做什么”与“仓库证据是否支持”之间的差距，当前仍属于启发式映射，需要在更多仓库和人工语义标注上继续验证。

后续计划：

1. 改进项目类型和非 Python 项目支持；
2. 增加更多 fresh holdout；
3. 接入 GitHub PR 评论和复现 badge；
4. 扩展到更多模型/数据/工作流生态；
5. 建立长期维护的开源基准。
6. 将复合 Action 升级为 GitHub App，支持组织级安装、Check Run 和自动评论。

附录

- 开源及第三方资源使用清单：docs/开源及第三方资源使用清单.md；
- AI 工具使用说明：docs/AI工具使用说明.md；
- 作品简介：docs/作品简介_300字.md；
- GitHub Action 使用指南：docs/GitHub-Action使用指南.md；
- 代码仓库：https://github.com/czkPK/reprolens；
- 标注文件：`repolens/benchmark/labels/`；

- 基准结果：repolens/benchmark/；
- 真实仓库报告：repolens/reports/。

附录 A 关键开源及第三方资源

资源	版本/来源	许可/授权	使用方式与边界
ReproLens 自主代码	v0.1.0 / GitHub	MIT	公开源码；保留版权与许可声明
Python 标准库	3.11+ / python.org	PSF	运行环境；不复制 Python 源码
Git	系统安装 / git-scm.com	GPL-2.0	网页版浅克隆；不随项目分发
GitHub API	REST API / GitHub	服务条款	读取公开仓库元数据；遵守限流
OpenAI-compatible API	使用者配置	服务条款	可选语义映射；无配置回退规则
GitHub Actions	checkout v4 / setup-python v5	MIT	CI 引用；保留声明
评测仓库	mem0、smolagents 等 19 个公开仓库	MIT/Apache-2.0/GPL-3.0/ELv2/自定义	仅作本地评测输入；不随项目分发源码
AI 编程助手	Codex/ChatGPT 服务	OpenAI 服务条款	开发辅助；人工测试、修改和证据核对

完整 28 项清单见仓库文件 docs/开源及第三方资源使用清单.md。第三方仓库源码和 autofix staging 副本不进入最终提交包。

附录 B AI 工具使用与人工审核

• AI 编程助手用于辅助代码起草、测试补充和文档结构整理；本人负责需求、方案、代码审查、测试、证据核对和最终修改。

• 运行时可选的 LLM 只负责语义声明映射和解释。返回结果必须引用已有检查项和证据 ID；未知 ID、格式异常或无 API 时自动回退规则模式。

• 仓库不保存 API Key、密码、访问令牌或个人敏感数据；网页版只分析公开 GitHub 仓库，默认不执行被审计仓库代码。

附录 C 快速复现实验

```
cd D:\CODEX\ReproLens\repolens
$env:PYTHONPATH = "src"
python -m unittest discover -s tests -v
python -m repolens audit examples/demo-llm-app --out-dir reports/demo-llm-app --autofix --explain
python -m repolens.web --open
```

测试预期：25 个测试通过；演示项目 49.1 -> 68.4（+19.3）；网页入口支持任意公开 GitHub 仓库。